

REQ2 – BedOfChaos

A. Growable Component Architecture

Design 1: Single Class with Internal Conditional Logic

Pros	Cons
Combining logic into one method reduces class count and initial setup.	Mixes parent and component creation logic, complicating maintenance.
Centralized growth logic ensures consistency and easier debugging.	Adding new component types requires modifying existing methods, hurting extensibility.
	Hard to reuse component behaviors independently due to tight coupling.

Design 2: Composite Pattern via a Growable Interface with Separate Classes

Pros	Cons
New component types can be added by implementing the Growable interface without altering existing classes.	Extra interfaces and classes can feel like boilerplate if behaviors are simple, increasing code size.
Clear separation: Leaf heals, Branch handles branching, BedOfChaos manages collections, improving clarity.	

B. BedOfChaos Actor and Behaviour Composition

Design 1: Hardcoded Priority Loop Over Behaviours

Pros	Cons
No Extra Abstractions Needed: Designers can see the behaviour order clearly, without needing a separate selector or strategy class, reducing initial complexity.	Adding a new behaviour or changing priority requires editing existing constructor logic.

	Other actors requiring similar behaviour logic must duplicate or refactor the loop.
--	---

Design 2: Strategy Pattern via a BehaviourSelector Interface

Pros	Cons
New selection strategies can be added without changing the actor class.	Adds abstraction overhead that may be unnecessary if only one strategy is used, increasing setup complexity.
The actor depends only on the BehaviourSelector abstraction, not on any specific selection algorithm, enhancing modularity.	Requires injecting a selector during actor creation, increasing setup complexity and risk of misconfiguration.
Selector logic can be reused across multiple actors, reducing duplication and supporting DRY principles.	

C. BedOfChaos Damage Point Calculation

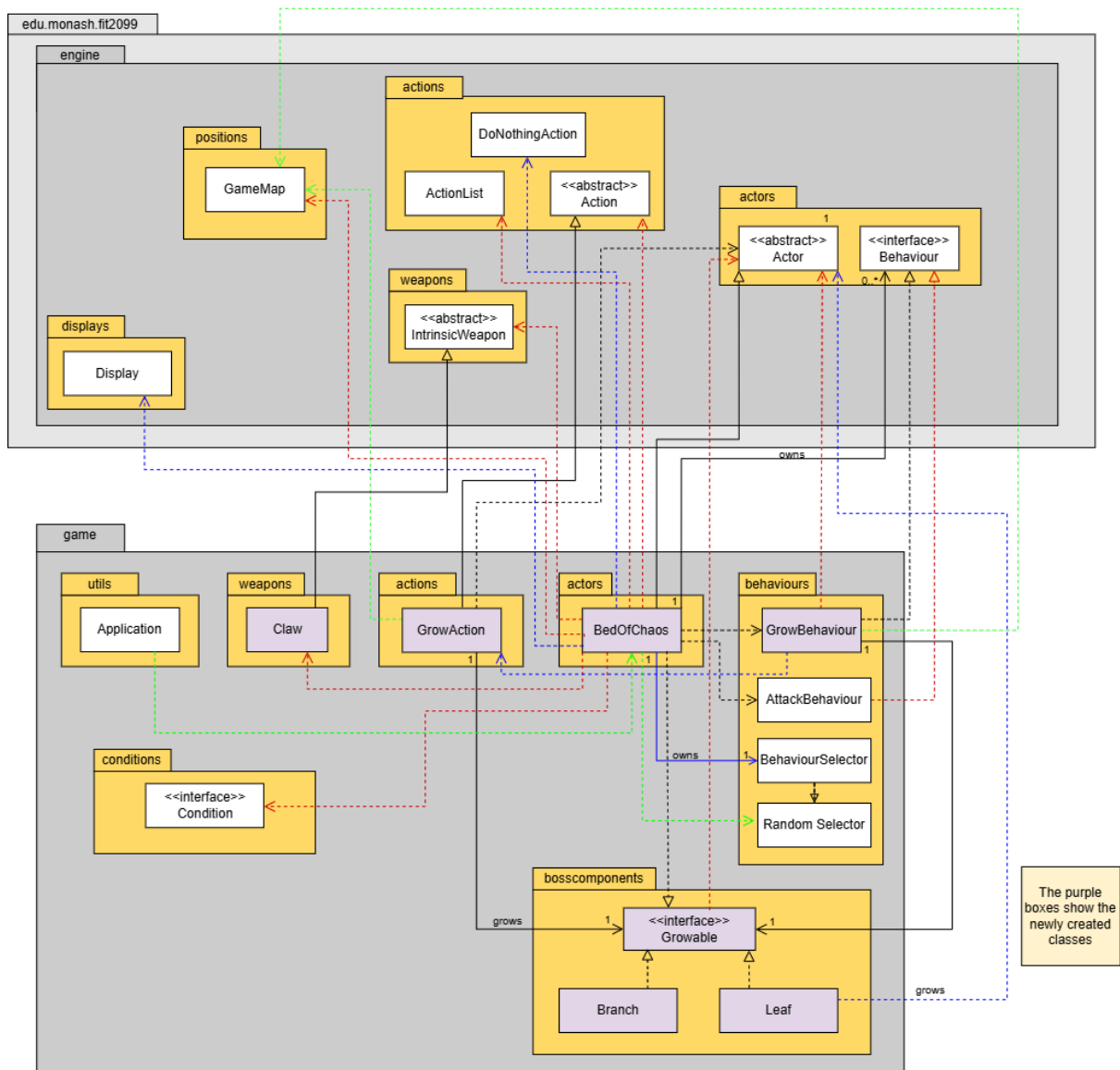
Design 1: Create 2 fields to keep track of the number of branches and leaves grown

Pros	Cons
Simple to understand and implement if there are only 2 components	Hard to reuse if new components are introduced due to tight coupling

Design 2: Use a list to store the components grown from BedOfChaos

Pros	Cons
Highly extensible as new components can be added without modifying existing code	Performance overhead as damage need to be recalculated every time
Allow damage point of its recursive components to be calculated recursively	

UML REQ2



The selected design for the **Growable Component Architecture** was Design 2, utilizing a Growable interface with specific implementations such as Branch, Leaf, and BedOfChaos. The **Behaviour Composition in BedOfChaos** follows Design 2, which uses a strategy pattern via a BehaviourSelector Interface. The **Damage Point Calculation for BedOfChaos** follows Design 2, which uses a list to store its components grown.

The reason we inject BehaviourSelection pattern inside BedOfChaos is to enhance the extensibility of our code, allowing behaviour to be selected with different strategy.

Each class focuses on one responsibility which strictly follows the **Single Responsibility Principle SRP**. Claw defines only weapon attributes like damage and verbs. Every Growable implementation handles its specific growth logic, while BedOfChaos aggregates them without accessing internals. Similarly, AttackBehaviour is solely concerned with deciding when to

attack, deferring condition checks to separate classes. `BedOfChaos.playTurn()` orchestrates the behaviour order but doesn't manage their individual logic.

Open/Closed Principle (OCP): The design supports extension without modifying existing code. New weapon variants like `FieryClaw` can subclass `Claw`. Additional growable components (e.g., `Thorn`, `Flower`) can implement `Growable` without affecting existing code. Behaviours and conditions can be expanded independently `LowHealthCondition` can be added to the system without altering existing structures when the boss only attacks when Player is low on health.

Liskov Substitution Principle (LSP): Subclasses like `FireClaw` or new `Growable` components can seamlessly replace their base types without breaking functionality. `AttackBehaviour` works with any `Condition` that meets the expected interface, preserving substitutability. `BedOfChaos`'s component list that stores `Growable` allow any `Growable` object to be used seamlessly in place.

Interface Segregation Principle (ISP): Interfaces are focused and minimal. `Condition` includes only `isSatisfied()`, and `Growable` exposes only necessary methods. This prevents classes from depending on methods they do not use.

Dependency Inversion Principle (DIP): High-level modules depend on abstractions, not concrete implementations. `AttackBehaviour` works with any `Condition`, and `BedOfChaos` manages a list of `Growable` interfaces. The sorted map of behaviours also decouples `BedOfChaos` from specific behaviour logic.

Don't Repeat Yourself (DRY): Logic is centralized and reused. Growth logic is encapsulated within each respective class, and `Claw` manages its attack attributes without needing duplication elsewhere.

Connascence Minimization:

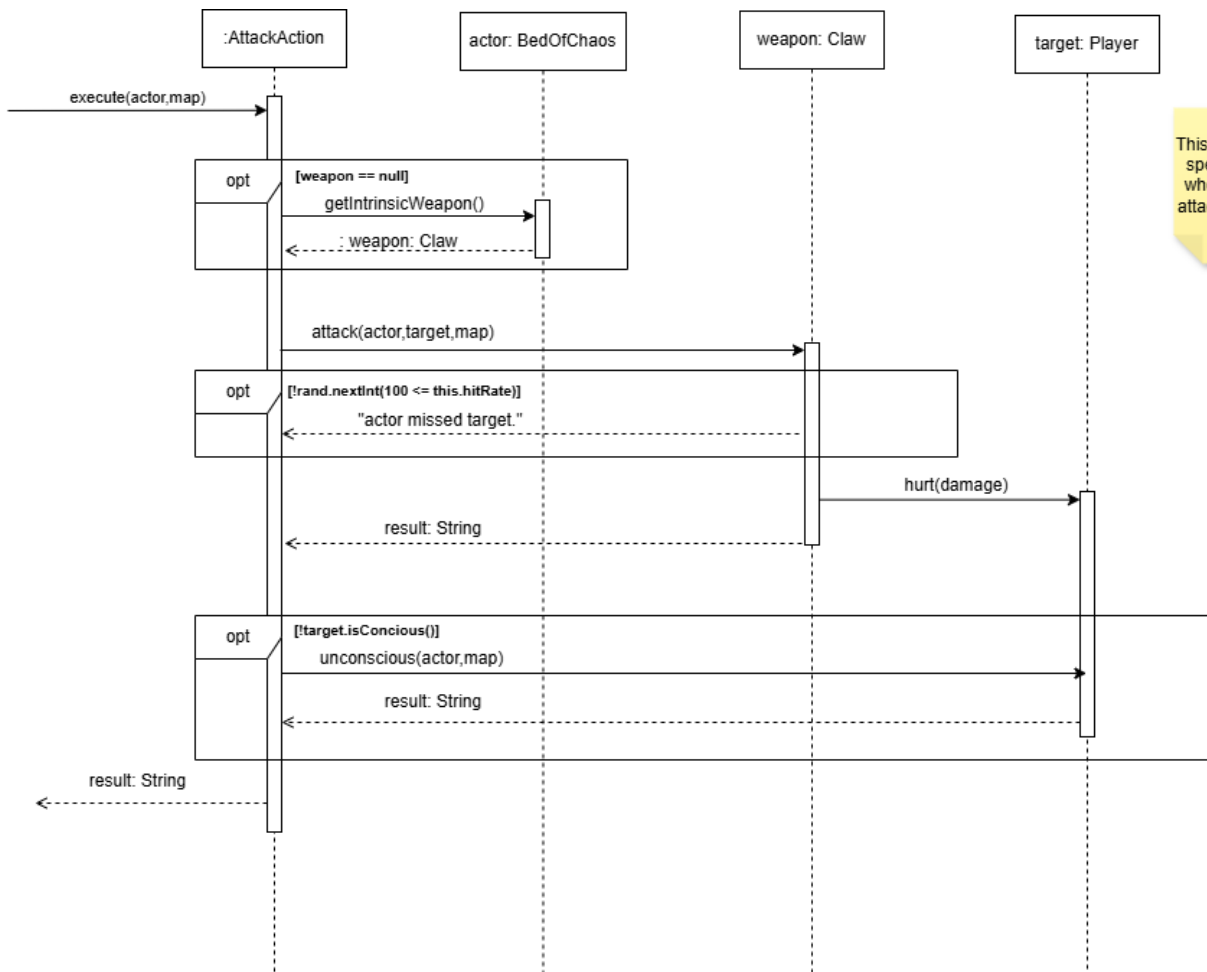
Classes communicate through interfaces rather than depending on internal structure. `BedOfChaos` does not know how each behaviour works, only that it provides an `Action`. `Growable` components function independently and are aggregated uniformly.

In this design, the primary types of **connascence** are **Connascence of Name (CoN)** and **Connascence of Type (CoT)**, as components such as `Growable`, `Condition`, and `Claw` rely on shared class and interface names and types to function correctly such as e.g., `BedOfChaos` interacts with `Growable` and `Behaviour` through well-defined type contracts. The design explicitly **minimizes stronger forms** like **Connascence of Meaning (CoM)** or **Connascence of Position (CoP)** by using abstraction boundaries and clearly named, minimal interfaces. For example, `BedOfChaos` does not depend on the internal workings of each `Growable` or `behaviour` class as it only depends on what their interfaces promise, promoting loose coupling and easier maintenance.

The system is highly extensible. New intrinsic weapons like PoisonClaw can be added by subclassing. Additional Growable types can be introduced with minimal impact. Composite conditions like “low stamina AND low health” can be created without touching existing behaviour logic.

By adopting a clean, modular structure with dedicated classes like Claw, Growable, and Condition, this design achieves strong separation of concerns. It adheres to SOLID principles and lays a robust foundation for future extensions, improvements, and gameplay evolution.

Sequence Diagram REQ2



This sequence diagram is specific to the scenario where the `BedOfChaos` attacks the player with its claw.